

캡스톤 디자인 · 최종발표

서울시 혼잡도 가이드

서울 관광지의 혼잡도를 단번에.
대체지까지 단번에.



팀

1조

팀원

김동현 · 이수진 · 권혁태

발표일

2026-05-27 (목)

지도교수

김하연 교수님

[GitHub · github.com/Dan-burn-go](https://github.com/Dan-burn-go)

목차

차별점과 실제 구현 증거 중심 · 10분 발표

01	배경 + 문제 정의	페르소나 설정, 서울 관광지 혼잡도 문제 파악
02	목표 + 차별점	기존 서비스와의 차별점과 기대효과
03	시스템 아키텍처	MSA + EDA · 기술 스택 한 장
04	핵심 기능 3종	서버 구조 + 실제 서비스 사진
05	모니터링 · 로그 · 분산 트레이싱	otel-lgtm 스택으로 운영 가시성-장애 추적 구축
06	진척 + 향후 계획	완료/진행/예정 · Phase 1~3
07	Q&A	질의응답

페르소나

다양한 사용자의 요구사항을 바탕으로 한 페르소나 구성



이름 : 권혁태

인물 정보 : 26세 남

소속 : 단국대학교 학생

거주지 : 용인시

행동 패턴 : 서울 나들이 주 1~2회

행동

단국대에서 2시간 걸려서 불꽃 놀이를 보러갔는데,
내가 가는 곳마다 그 날 사람이 제일 많아..

혼잡도를 미리 알 수 있었다면,
가장 덜 혼잡한 지역을 찾아서 불꽃 놀이에 집중할 수 있었을텐데.
기존 서비스는 시각적으로 보기에 안 좋고, 대체지역을 찾기도 어렵고,
확인했더라도 경로 탐색은 다른 앱을 이용해야해서 불편해.
이런 불편함을 해결할 수 있는 서비스 어디 없나?

요구사항

- 지도에 떠 있는 혼잡도를 미리 확인하고 이동할 수 있다.
- 혼잡한 장소 연관도 높은 대체지를 파악할 수 있다.
- 분산된 정보(경로, 문화행사) 등을 한 곳에서 파악할 수 있다.

배경 + 문제 정의

서울시 관광지 122곳 혼잡도 공개 API 제공 · 일반 사용자용 서비스 부재

122곳

서울 주요 관광지

5분 주기

공개 API 갱신

JSON

응답 포맷

무료 · 공공

이용 조건

01

혼잡도 접근성

공공 API는 있지만
일반 사용자가 쉽게 보는
서비스 부재

02

원인 설명 부재

단순 수치만으로는
"왜 붐비는지"
알 수 없음

03

대안 탐색 분산

대체지/교통을 찾으려면
여러 앱을 번갈아
사용해야 함

기존 지도 서비스에 없던 세 가지

붐비는지 알려주는 데서 끝나지 않고, 왜 붐비는지와 어디로 갈지까지.

기존 서비스

혼잡 여부까지만

× 네이버 · 카카오 지도

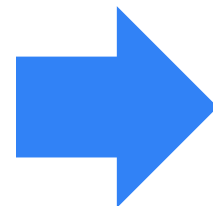
일부 상점 단위만 혼잡도 제공

× 구글 Popular Times

관광지 혼잡도는 보여주지만 그걸로 끝

× 서울 실시간 도시데이터

데이터는 공개되지만 사용자 화면은 없음



단번에 — 더한 세 가지

01

AI 원인 분석

혼잡한 이유를 AI가 자연어로 설명

02

대체지 추천

반경 2km 안 덜 붐비는 유사 장소 제안

03

교통 경로 통합

대체지까지 가는 길을 소요시간 순으로 안내

기대효과

혼잡도를 미리 알고, 이유를 이해하고, **단번에** 해결한다.

단번에 — 더한 세 가지

01

불필요한 이동 감소

목적지 도착 전에 혼잡도와 원인을 미리 파악함으로써, 헛걸음을 줄일 수 있다.

02

혼잡 원인 인지로 인한 능동적 의사결정

AI가 분석한 원인(공연·행사·시간대 패턴)을 함께 제공함으로써 사용자가 "피할지, 즐길지"를 스스로 판단하는 것이 가능하다.

03

정보 탐색 비용 절감

혼잡도 확인 → 원인 파악 → 대체지 탐색 → 경로 검색까지 지도 기반의 직관적인 화면을 가진 단일 서비스에서 해결할 수 있다.

전체 시스템 아키텍처

MSA System Architecture
인파밀집사고 위험 감지 & 혼잡도 분석 시스템

The diagram illustrates the system architecture for the In-Population Density Accident Risk Detection and Congestion Analysis System. It is divided into two main Docker Clusters.

Frontend Cluster:

- CLIENT** (User) sends a **USER Request** to **Vercel** (React).

Main Backend Cluster:

- API Gateway** receives a **Backend Request** from Vercel and routes it to three services, all running on **spring boot**:
 - Mobility Service**
 - Congestion Analysis Service**
 - Map Service**
- Data Flow and Services:**
 - Congestion Analysis Service** interacts with **Mysql** (for congestion analysis requests when density exceeds a threshold) and **RabbitMQ** (for congestion data).
 - Map Service** interacts with **Mysql** (for map information) and **redis** (for map information caching).
 - FastAPI AI Analysis Service** receives **분석 결과 반환** (analysis result return) from RabbitMQ and sends **혼잡도 데이터** (congestion data) back to the Congestion Analysis Service.
 - redis** provides **문화정보 캐싱** (cultural information caching) and **혼잡도 데이터 캐싱** (congestion data caching) to the Map Service.

Monitoring Cluster:

- Contains **Grafana/otel-lgtm**, **Grafana**, **Prometheus**, **Grafana loki**, and **OpenTelemetry**.
- It receives **이상시 알림 전송** (notification transmission in case of abnormality) and sends alerts to **slack**.

Data Source:

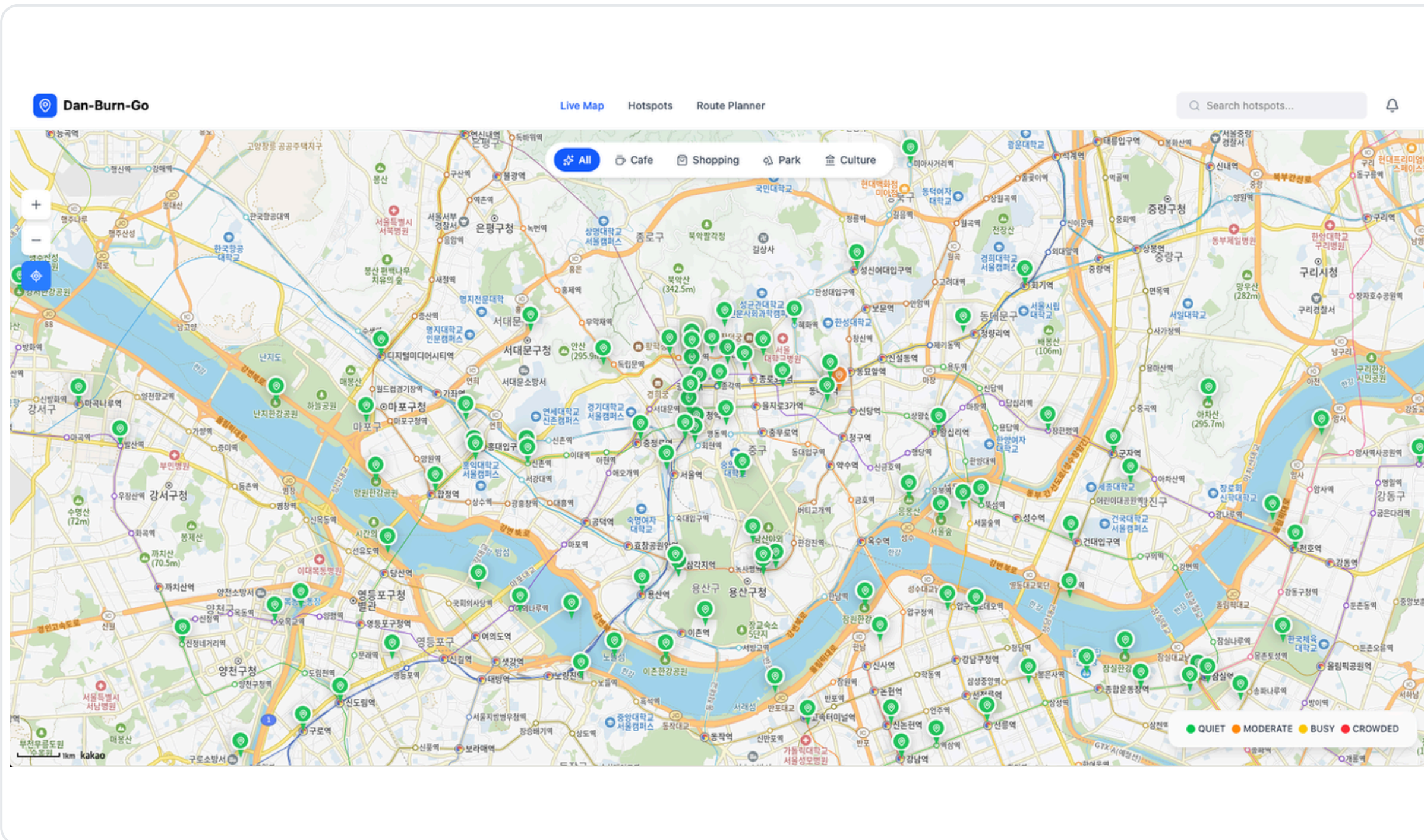
- 서울시 공공 데이터** (Seoul City Public Data) provides data to the Map Service.

Prometheus · Grafana · Loki · Tempo

핵심기능 1 — 사용자 화면

실시간 혼잡도 수집 — 프론트 UI

서버가 수집한 데이터 · goseoul.today 에서 사용자에게 공유



구현 완료

메인 지도 페이지

카카오맵 + 마커

서울 관광지 122곳 혼잡도

카테고리 필터

관심 장소 분류 조회

혼잡도 4단계

색상 범례 제공

5분 주기

마커 색상 자동 갱신

goseoul.today

src/ 계층 분리 및 역할 분담

각 레이어의 명확한 역할 분담을 통한 유지보수성 향상

src/ 아키텍처

api/ - HTTP 호출 전달

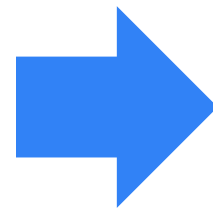
hooks/ - 비즈니스 로직

components/ - UI 렌더링

pages/ - 화면 조립

types/ - 전역 타입 정의

data/ - 정적 좌표 데이터



01

Backend API

↓ Axios 호출(api/레이어)

02

useCongestionMarkers

↓ 비즈니스 로직 처리(hooks/레이어)

03

Kakao Map 렌더링

↓ ConjestionMarker + 121개 인터렉션

좌표 데이터는 왜 프론트에서 관리하나?

Problem

서울시 공공 API는 areaCode + 혼잡도만 제공 — 지도 렌더링에 필수적인 위경도 데이터 누락

OPTION A

백엔드 DB 저장

좌표 데이터를 서버 DB에 저장하고 혼잡도와 Join하여 API로 함께 제공하는 방식

× 불변 데이터 전송 낭비

OPTION B

Places API 실시간 검색

카카오/구글 장소 검색 API를 통해 매 호출마다 실시간으로 좌표를 획득하는 방식

× Rate Limit & 응답 지연

OPTION C

프론트엔드 Static 관리

바뀌지 않는 121개의 좌표를 JSON/Static 파일로 포함하여 즉시 조회하는 방식

✓ Zero Latency & 고성능

실시간 데이터 업데이트 구현 전략

백엔드 배치 데이터 구조와의 정렬 및 프론트엔드 최적화

STRATEGY 01

왜 폴링(Polling)을 선택했는가?

❌ WebSocket / SSE 지양

- 5분 배치 구조와 서버 Push 방식의 불일치
- 인프라 비용 및 백엔드 추가 공수 발생

✅ Polling 방식 채택 [Efficiency]

- 프론트엔드 단일 수정으로 즉시 대응
- 기존 API 재사용을 통한 구현 안정성

STRATEGY 02

업데이트 주기(1분)의 근거

⌚ 5분 주기 폴링 시 **Worst**

- 데이터 갱신 직후 요청 시 **최대 10분 지연**

⚡ 1분 주기 폴링 시 **Optimal**

- 데이터 반영 체감 속도 극대화
- Redis 캐싱 적용으로 **서버 부하 미미**

STRATEGY 03

불필요한 리렌더링 차단

1분 간격 API fetch()

▼

populationTime 데이터 대조

▼

변경 시에만 setMarkers() 호출

👁️ 마커 깜빡임(Flicker) 현상 해결

⚙️ 브라우저 렌더링 자원 최적화

반응형 UX 대응 및 향후 로드맵

↕ ① 지도 높이 (View-port)

100vh ✗

주소창 간섭으로 하단 UI 잘림

h-dvh ✓

Dynamic Viewport 대응

📱 ② 반응형 컴포넌트

Mobile (md 미만) **Drawer Menu**

햄버거 아이콘 인터랙션

Desktop (md 이상) **Expanded Header**

검색바 및 전체 메뉴 노출

☰ ③ 혼잡도 범례 최적화

Mobile (좌하단) 약자 표기

Q M B C

Desktop (우하단) 전체 명칭

QUIET ~ CROWDED

🗺️ Project Roadmap

✓ 완료 (Done)

- 지도 엔진 및 실시간 혼잡도 마커
- 카테고리 필터링 및 데이터 폴링

🔄 진행 중 (WIP)

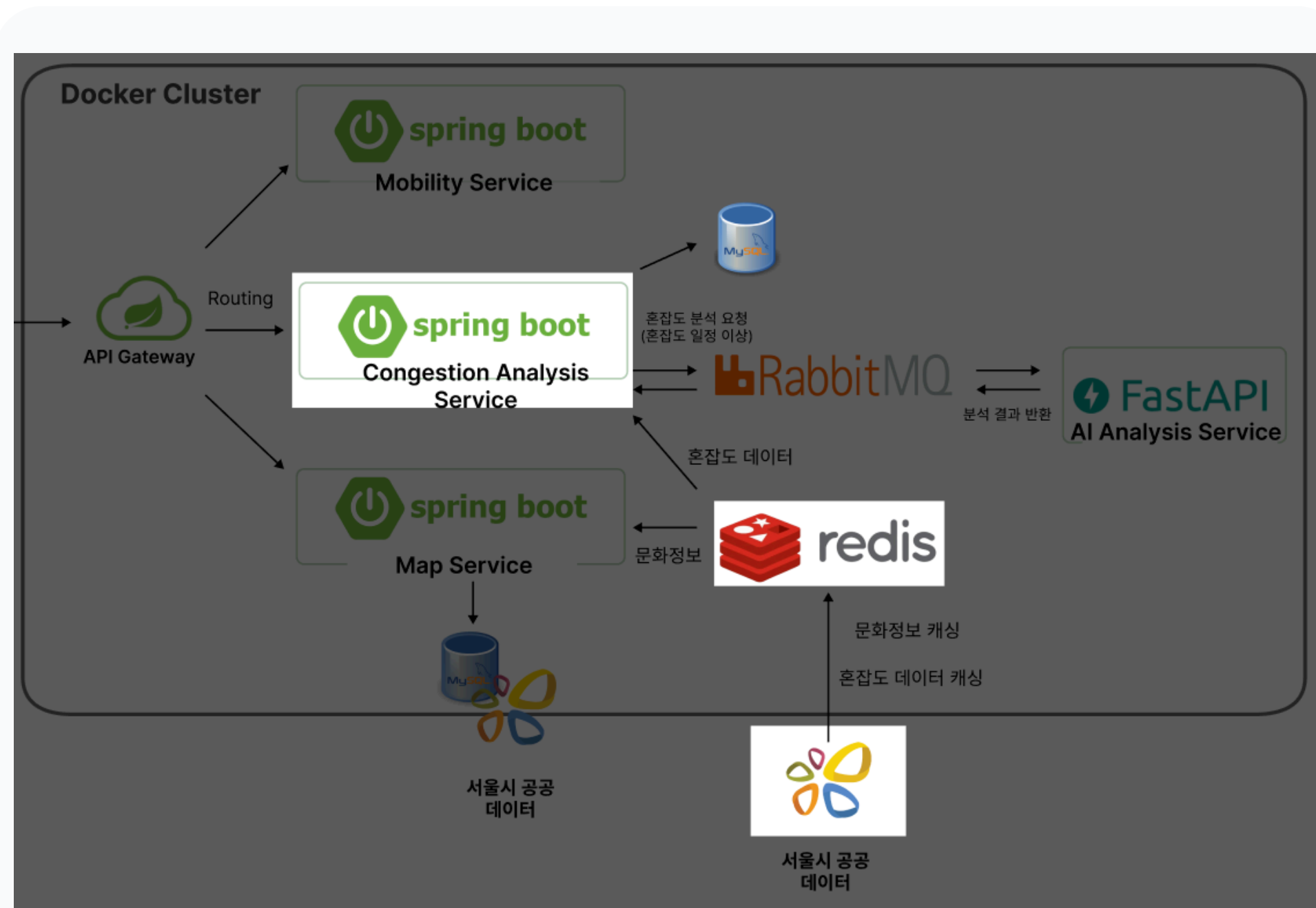
- 대체지 추천 엔진 API 연동

📅 예정 (Next)

- **Detail Page:** AI 분석 결과 통합
- **Ranking Page:** 지역별 혼잡도 순위
- **Marker:** 대량 마커 클러스터링
- **Optimization:** React Profiler & Lighthouse
- **UX:** MapControls 핸들러 연결

실시간 혼잡도 수집 — 서버 구조

주기적으로 수집해 Redis에 저장, MySQL에는 비동기로



수집 흐름 · 4단계

- 1 5분 주기 스케줄러**
서울시 공공 API → 122개 장소
- 2 Redis Pipeline**
Write-behind, Look-aside
- 3 MySQL 비동기 저장**
@Async · 수집 경로 블로킹 없음
- 4 매일 새벽 3시 정리**
7일 지난 데이터 자동 삭제

기술 포인트

- Redis Pipeline — Write-behind, Look-aside
- Stub Client — API Key 없는 환경에서도 개발/CI 보장

실시간 혼잡도 수집 — 로그

5분 주기 수집 사이클이 로그로 실제 동작 확인

```
⋮ 2026-04-23 11:39:02.856  DEBUG  app=service-congestion host=a17b06116d18 service_name=service-congestion 2026-04-23 02:39:02.856 DEBUG [service-congestion,367a11e71b44c1deae47223c5466ae2,d4b4a05eaada9fe0] --- [      async-db-2] c.d.c.service.CongestionService          : [CongestionService] DB 이력 저장 완료 - 121건

⋮ 2026-04-23 11:39:02.752  INFO   app=service-congestion host=a17b06116d18 service_name=service-congestion 2026-04-23 02:39:02.752  INFO [service-congestion,ed2d8ba7115a0cde7220151023dd58a,773641edaa29a59a] --- [    scheduling-1] c.d.c.scheduler.CongestionScheduler      : [CongestionScheduler] 혼잡도 데이터 수집 완료 - 121/121건

⋮ 2026-04-23 11:39:01.787  INFO   app=service-congestion host=a17b06116d18 service_name=service-congestion 2026-04-23 02:39:01.787  INFO [service-congestion,ed2d8ba7115a0cde7220151023dd58a,773641edaa29a59a] --- [    scheduling-1] c.d.c.scheduler.CongestionScheduler      : [CongestionScheduler] 혼잡도 데이터 수집 시작
```

LOG

수집 스케줄러

5분 주기 실행

LOG

혼잡도 수집

121개 장소 완료

LOG

DB 이력 저장

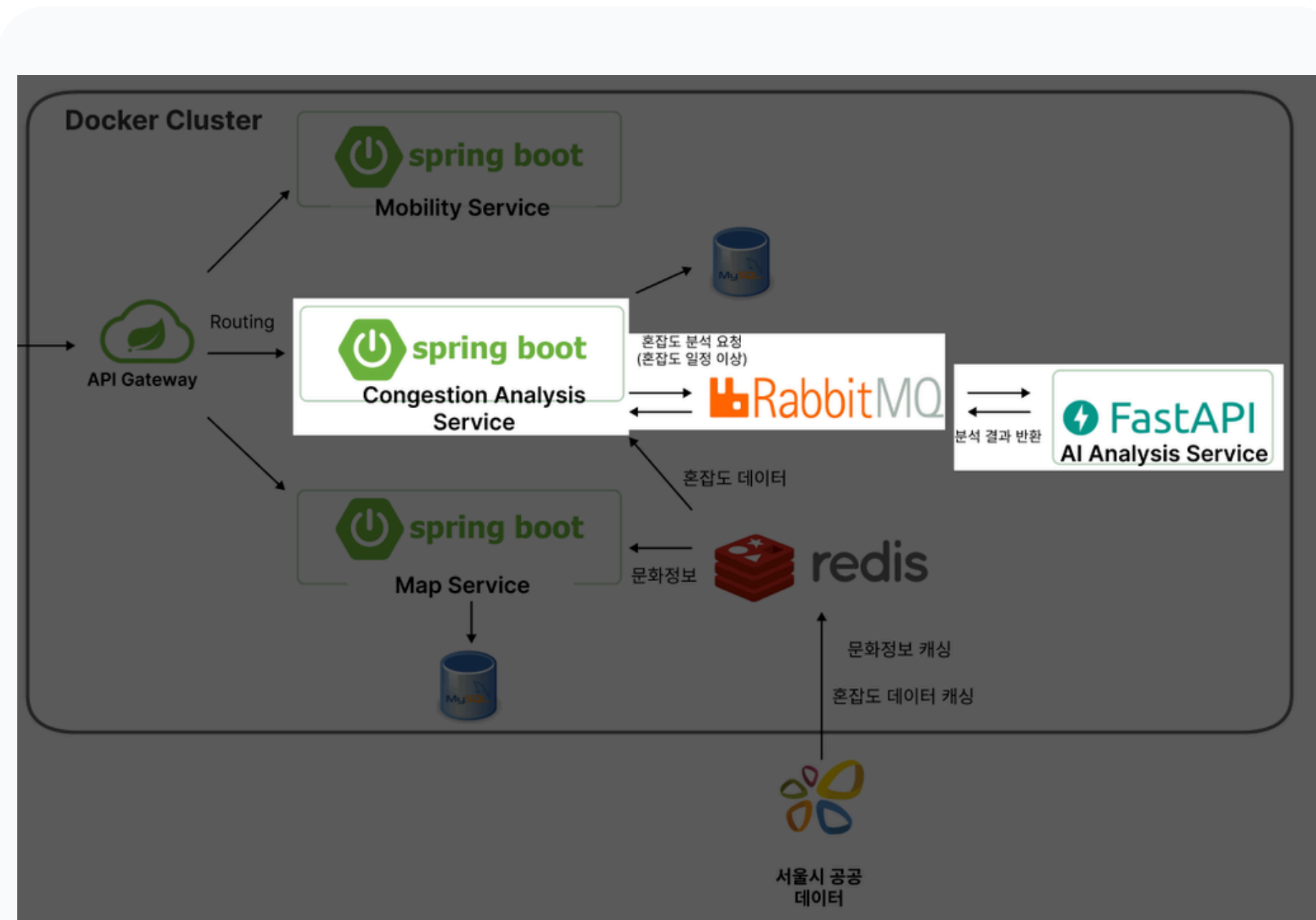
121건 INSERT 완료

END-TO-END

감지 → 발행 → 배치 → AI 호출 → 저장
전 구간 로그로 검증됨

AI 원인 분석 — 이벤트 기반 구조

Java 수집 서버와 Python AI 서버를 RabbitMQ로 분리



WHY EDA?

5분 주기 버스트 × AI API Rate Limit 충돌

→ RabbitMQ로 분리 · 장애 격리 · 배치 윈도우로 LLM 비용 절감

이벤트 흐름 · 6단계

- 1 Congestion Service: BUSY 상승 전이 감지
- 2 congestion.events로 메시지 발행
- 3 AI Service: 10초 배치 윈도우 수신
- 4 AI 모델 1회 호출로 다건 분석
- 5 분석 결과 역방향 이벤트 발행
- 6 Congestion Service → Redis 캐싱 + MySQL 저장

[Blog · velog.io/@kim138762](https://velog.io/@kim138762)

AI 원인 분석 — 모니터링 · 로그

프론트 UI는 구현중 · 서버는 아래 흐름대로 실제 동작

```
: 2026-04-20 19:59:03.989 INFO app=service-congestion host=a17b06116d18 service_name=service-congestion 2026-04-20 10:59:03.989
0-1] c.d.c.event.AiReportEventConsumer : [AiReportConsumer] AI 분석 결과 저장 완료 - areaCode=POI070

: 2026-04-20 19:59:03.978 INFO app=service-ai logger=app.rabbitmq.publisher service_name=service-ai severity=info 2026-04-20 10:59:0
lisher] AI 분석 결과 발행 - 1건
```

GRAFANA

AI 호출 수

시간대별 Bar

GRAFANA

배치 크기 분포

Histogram

GRAFANA

LLM 응답 시간

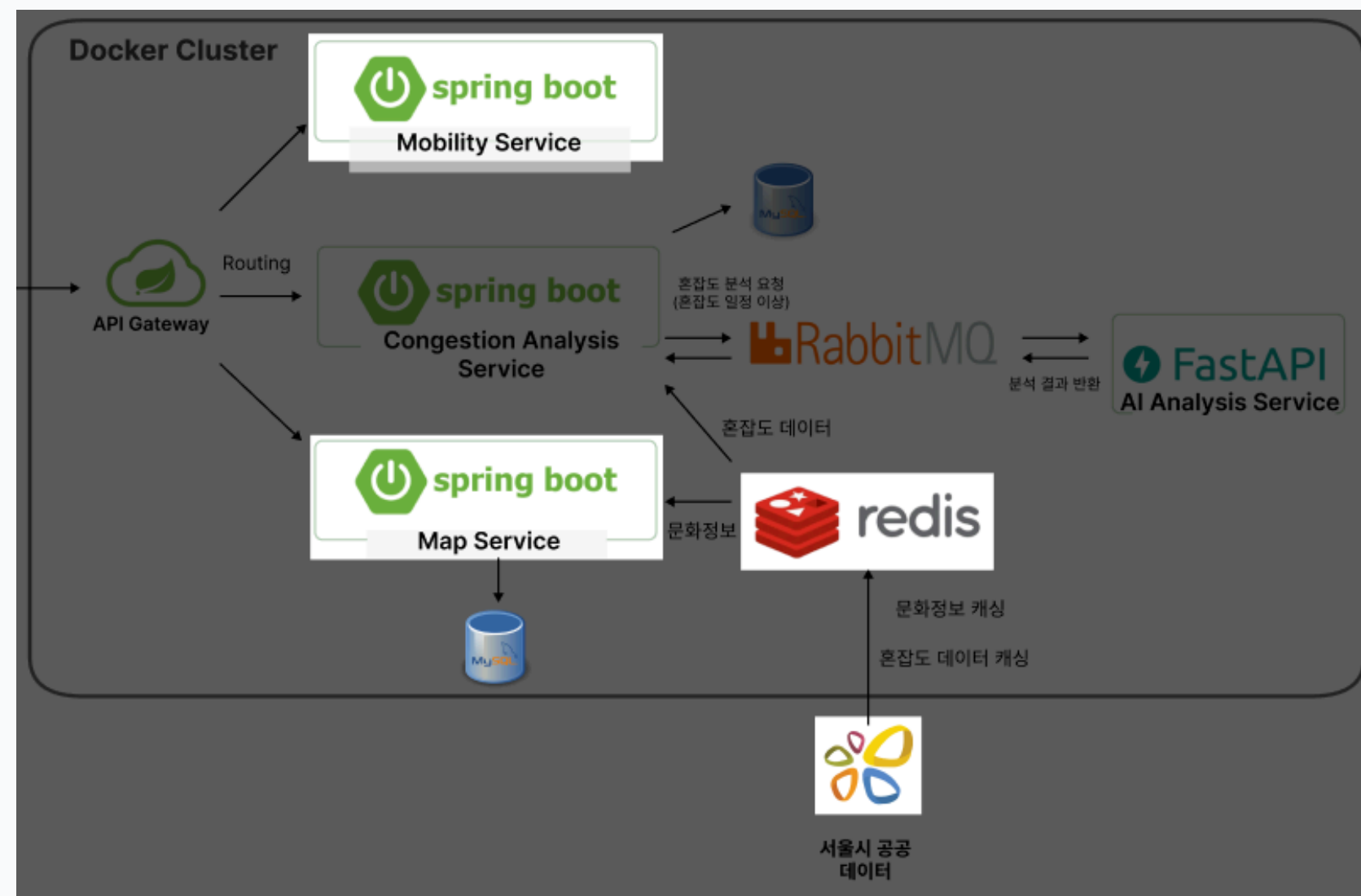
p50 / p95 · Line

END-TO-END

감지 → 발행 → 배치 → AI 호출 → 저장
전 구간 로그로 검증됨

대체지 · 교통 경로 추천 — 서버 구조

혼잡 지역의 대체지와 가는 방법



대체지 추천

Map Service

공간 쿼리

반경 2km 후보 추출

카테고리

원 장소와 유사 분류만 매칭

혼잡도 반영

후보의 현재 혼잡도 함께 반환

교통 경로

Mobility Service

외부 API

길찾기 · 버스 API 연동

정렬

소요시간 순

결합 시

현재 위치 → 덜 붐비는 대체지

기술 포인트

- ST_Distance_Sphere · DB 레벨 공간 쿼리
- 후보 N개 소요시간 · 외부 API 병렬 호출
- 거리 + 혼잡도 반영한 추천

핵심기능 3 — 추천 서버

대체지 · 교통 경로 추천 — 구현 사진

혼잡 지역의 대체지와 가는 방법



대체지 추천

Map Service

지도에서 대체지역 버튼

카테고리, 거리 기반 점수를 통해서
대체지역을 추천

대체지역은 총 3곳을 추천

원 장소와 유사 분류만 매칭

혼잡도 반영

후보의 현재 혼잡도 함께 반환

핵심기능 3 — 추천 서버

대체지 · 교통 경로 추천 — 구현 사진

혼잡 지역의 대체지와 가는 방법

실시간 혼잡도 안내

어디로 가볼까요?

핫플레이스까지 가는 가장 빠른 길,
도착지의 지금 분위기와 함께 알려드려요.

경로 입력

출발지

단국대학교 죽전캠퍼스

도착지

강남역

경로 찾기 →

도착지 · 문화

강남역

약간 붐빔

현재 71,000명

경로 8개 · 빠른 순

최단 경로

39 분

4,750원 · 환승 2회

8100 / 8100(예약)

신분당선

● 단국대학교 죽전캠퍼스 출발

도보 3분

● 단국대정문

8100 / 8100(예약)번 16분

15분 간격

● 정자역

도보 2분

● 정자

신분당선 15분

대체지 추천

Mobility Service

출발지와 도착지 선택

혼잡도를 확인하고 경로추천으로 연결

가장 빠른 경로부터 추천

ODsay API를 통한 정확한 경로

대중 교통 API를 활용

대체지 · 교통 추천 — 로그

대체지 추천 로직 개발 중 · 문화행사 수집 스케줄러 구현 완료

기반 데이터 파이프라인

LIVE

Map Service 스케줄러

- 문화행사 API 호출
- 125 items DB INSERT (2.3s)
- 매시간 실행 후 문화 행사 목록 동기화

WIP

추천 비즈니스 로직

- 공간 쿼리 + 혼잡도 정렬
- 외부 API 병렬 호출

: 2026-04-20 23:00:01.275	INFO	app=service-map host=091c9b33468e service_name=service-map	2026-04-20 14:00:01.275
duler.EventSyncScheduler	:	[Scheduler] 주간 서울시 문화행사 동기화 종료	
: 2026-04-20 23:00:01.275	INFO	app=service-map host=091c9b33468e service_name=service-map	2026-04-20 14:00:01.275
map.service.EventService	:	문화행사 동기화 및 기간 만료 데이터 삭제 완료	
: 2026-04-20 23:00:01.265	INFO	app=service-map host=091c9b33468e service_name=service-map	2026-04-20 14:00:01.265
ice.EventUpsertService	:	배치 처리 완료 - Insert: 0건, Update: 5건	
: 2026-04-20 23:00:00.938	INFO	app=service-map host=091c9b33468e service_name=service-map	2026-04-20 14:00:00.938
ice.EventUpsertService	:	배치 처리 완료 - Insert: 0건, Update: 5건	
: 2026-04-20 23:00:00.696	INFO	app=service-map host=091c9b33468e service_name=service-map	2026-04-20 14:00:00.696
ice.EventUpsertService	:	배치 처리 완료 - Insert: 0건, Update: 13건	
: 2026-04-20 23:00:00.464	INFO	app=service-map host=091c9b33468e service_name=service-map	2026-04-20 14:00:00.464
ice.EventUpsertService	:	배치 처리 완료 - Insert: 0건, Update: 394건	
: 2026-04-20 23:00:00.005	INFO	app=service-map host=091c9b33468e service_name=service-map	2026-04-20 14:00:00.005
map.service.EventService	:	서울시 문화행사 API 동기화 시작	

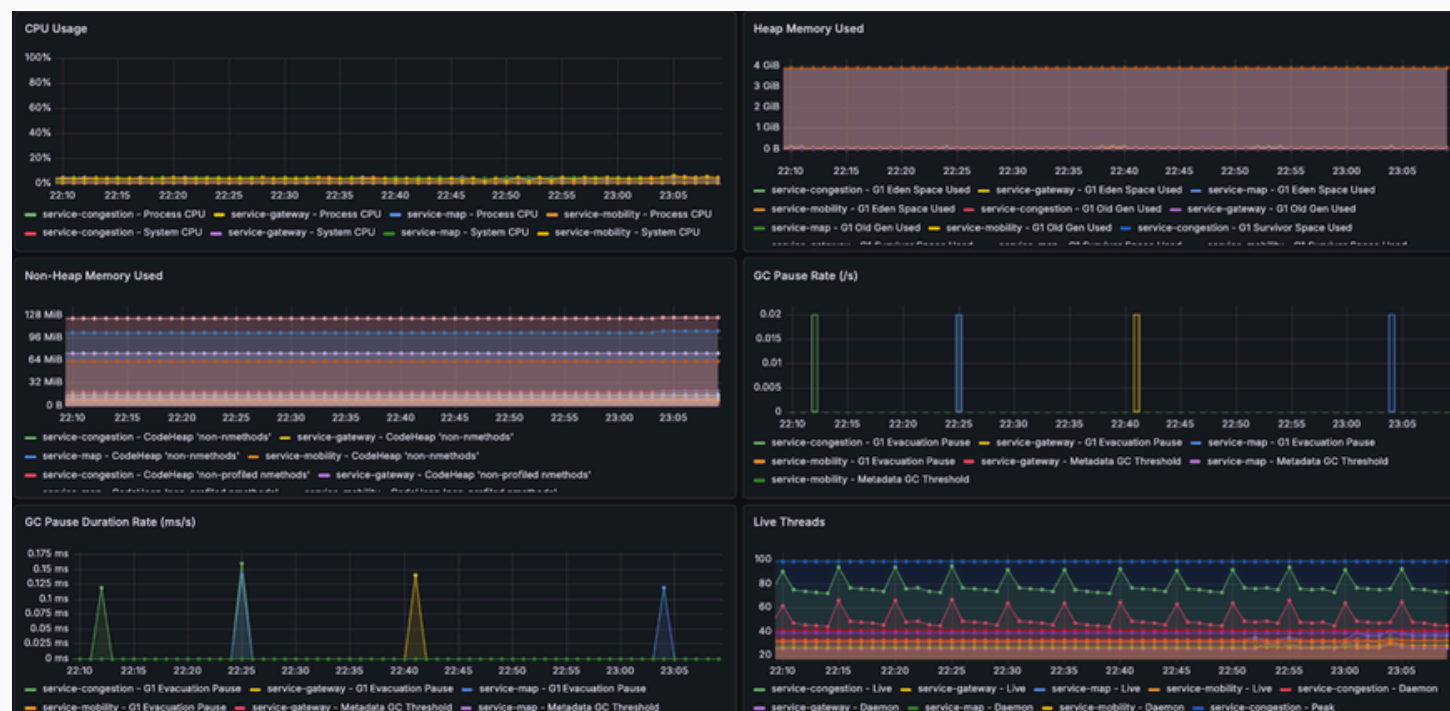
05 · 관측성 — 메트릭

시스템 지표 · 대시보드

Prometheus로 지표 수집 · Grafana로 JVM · API 실시간 확인

METRICS

JVM · 메모리 · CPU



Spring Actuator · FastAPI instrumentator 로 수집

METRICS

API 요청 · 응답 시간



서비스별 요청 수 · 에러율 · p95 응답 시간

otel-lgtm 스택

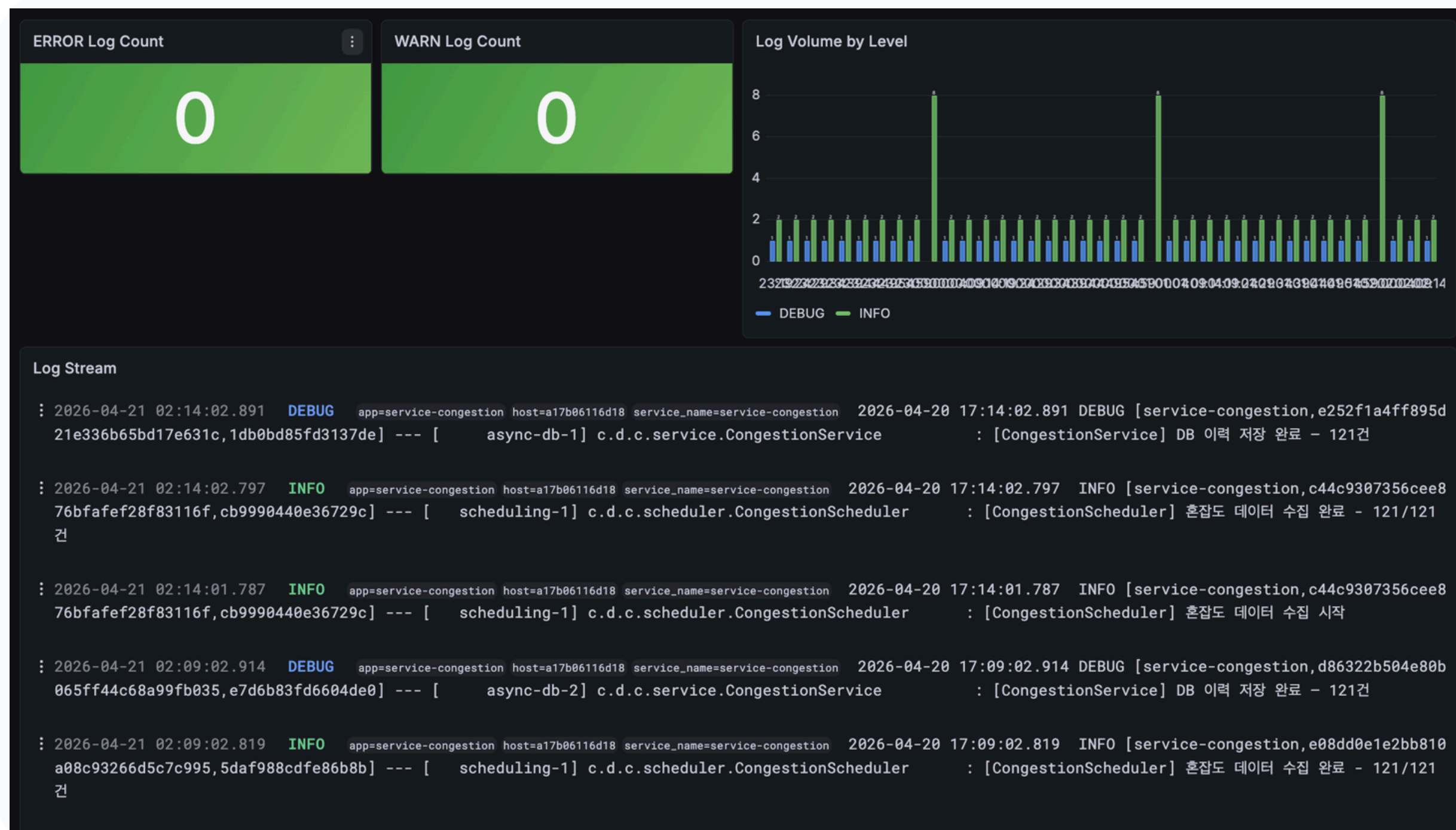
Prometheus · 지표 수집

Grafana · 대시보드

다음 장 → Grafana Loki · 구조화 로그

구조화 로그 — Grafana Loki

Loki4j로 JSON 로그 전송 · Grafana에서 레벨 · 서비스별 확인



구성 요소

Loki4j logback

Spring 로그 전송

JSON 구조화

서비스 · 레벨 태깅

Grafana Loki

집계 · 검색 · 시각화

대시보드가 보여주는 것

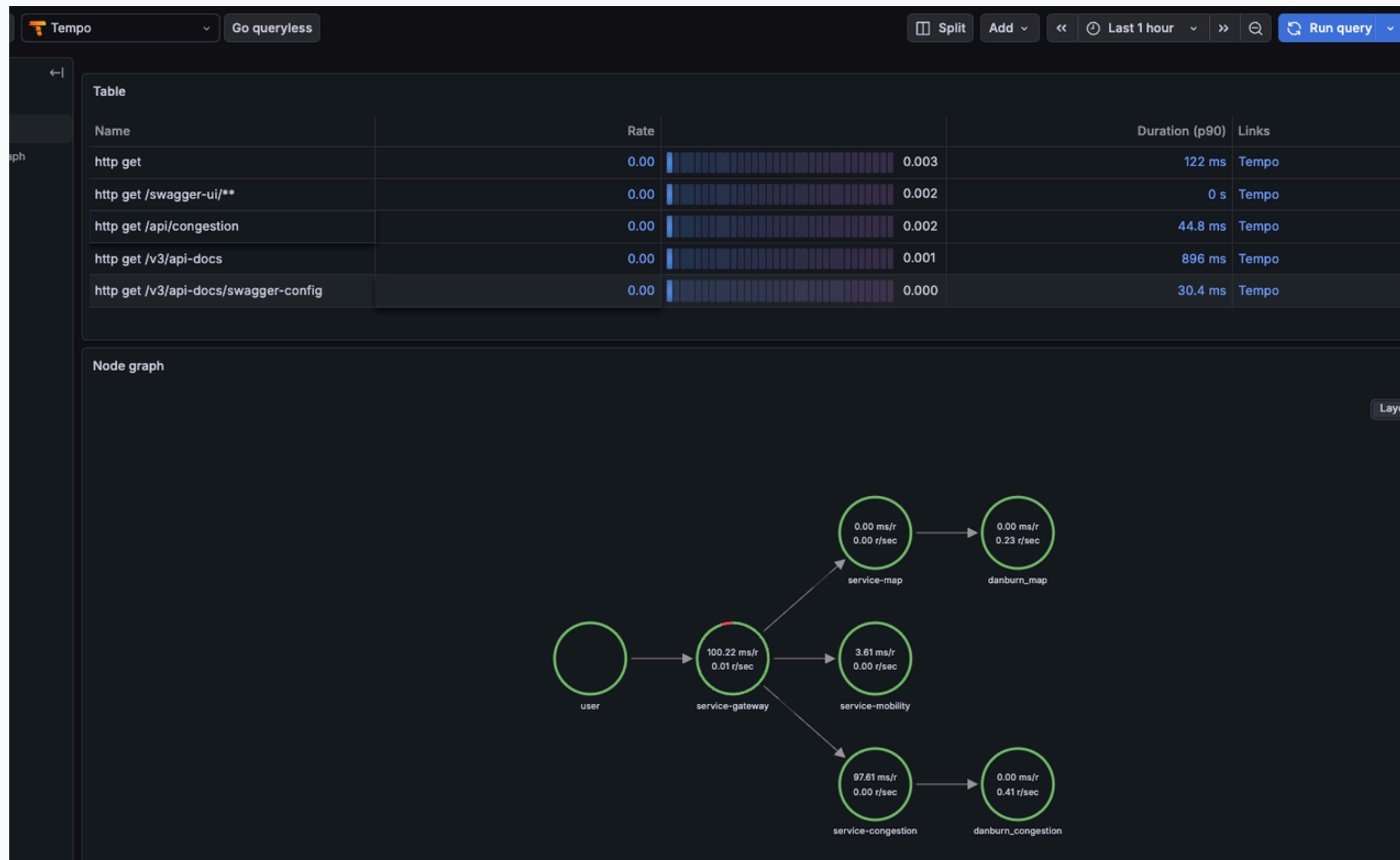
· ERROR · WARN 카운트 — 이상 즉시 감지

· 레벨별 로그 볼륨 추이 — 배포 영향 관찰

· 실제 로그 스트림 — 혼잡도 수집 · 스케줄러 흐름

요청 흐름을 하나의 Trace ID 로 추적

OpenTelemetry로 측정 · Grafana Tempo에 저장 · 서비스 간 호출 확인



구성 요소

Micrometer Tracing

Spring 계측

OpenTelemetry (OTLP)

표준 전송

Grafana Tempo

Trace 저장 · 조회

이 트레이싱으로 할 수 있는 것

- Gateway → 서비스 → DB · 외부 API 호출을 한 Trace ID 로 연결
- 특정 요청이 어느 단계에서 얼마나 걸렸는지 1회 조회로 파악
- 장애 · 병목이 발생한 지점을 빠르게 좁혀서 원인 파악

06 · 진척 현황

진행 현황

제출 시점 진행중인 내용, 원래 프로젝트 완료 계획이 6월 초

DONE

완료

- MSA + EDA 아키텍처 설계
- Spring Cloud Gateway 구성
- 혼잡도 수집 (Redis + MySQL)
- RabbitMQ EDA + DLQ
- AI Service (LLM 배치 호출)
- 프론트 메인 지도 + CI/CD
- otel-lgtm 관측성 스택
- Mobility Service 경로
- 장소 상세 페이지
- 혼잡도 랭킹 페이지
- Map Service 대체지역 추천

WIP

진행 중

- k6 부하테스트
- Virtual user 테스트
- 실사용자 검증

백엔드

인프라 · 수집 · AI · 관측성 완료 / Map · Mobility 로직 진행

95%

프론트엔드

상세 · 페이지 구현중

95%

- Circuit Breaker (Resilience4j)
- 부하 테스트 (k6)

- MCP + Function Calling
- 배포 환경 최종 구축

06 · 향후 계획

향후 계획

남은 기간 안정화 및 고도화

PHASE 2

안정화 · 고도화

Hardening

k6 부하 및 VU 테스트

감사합니다.